Assignment #4 Satellite Image Classification Using Transfer Learning

測量115 F64111134 施淳譯

1. **Briefly describe the Model #1-#8 in the code, and show the performances of them.**

Model #1: A Fully Connected Network (FCN) with a single dense layer. The dense layer has an input tensor of (4096, 1) and an output tensor of (10, 1). The total parameters include 4096 × 10 weights and 10 biases. A softmax activation function is used to output the classification probabilities. The classification accuracy of the testing data is 24.97%.

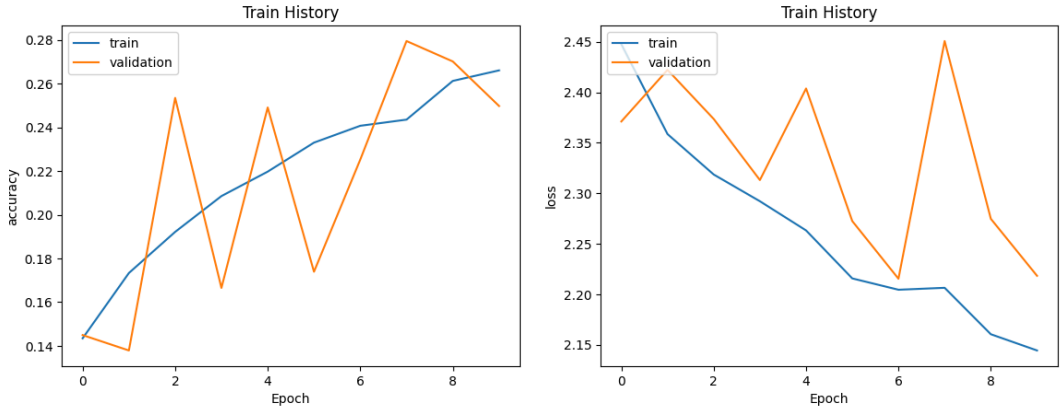


Figure 1-2. Training and Validation Result of Model #1

Model #2: An FCN with two dense layers. The first layer has an input tensor of (4096, 1) and an output tensor of (256, 1), followed by a ReLU activation. The second layer has an input tensor of (256, 1) and an output tensor of (10, 1), using a softmax activation. The first layer includes 4096 × 256 weights and 256 biases, while the second layer includes 256 × 10 weights and 10 biases. The classification accuracy of the testing data is 27.68%.

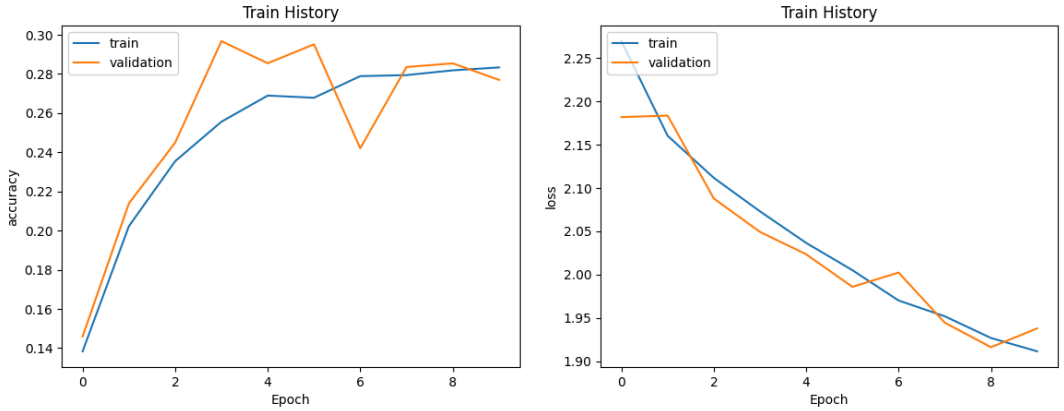


Figure 3-4. Training and Validation Result of Model #2

Model #3:An FCN with four dense layers and three dropout layers. The first three dense layers each have an output tensor of (256, 1) with ReLU activation, and each is followed by a dropout layer with a dropout rate of 20%. The last dense layer outputs (10, 1) with softmax activation. The first dense layer has an input tensor of (4096, 1), while the next three have (256, 1). The total number of trainable parameters is 1,182,986. The classification accuracy of the testing data is 15.30%.

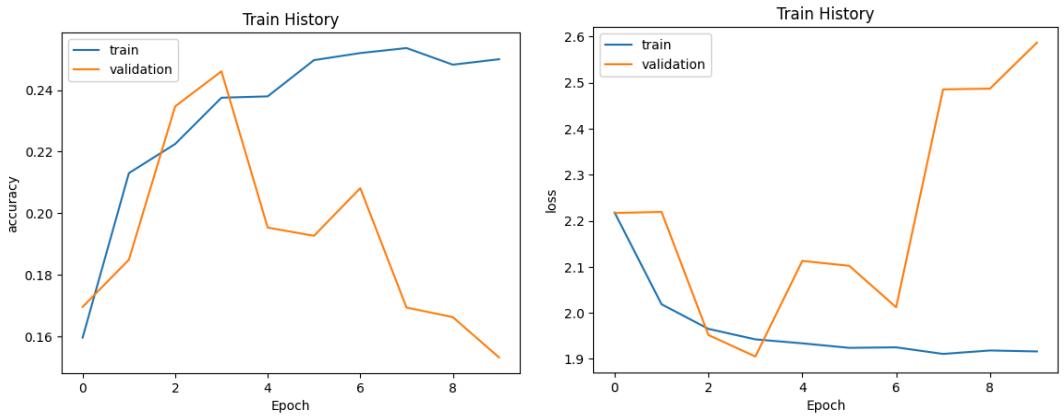


Figure 5-6. Training and Validation Result of Model #3

Model #4: A Convolutional Neural Network (CNN) architecture built with convolutional, max-pooling, dropout, and dense layers. The first two convolutional layers use (3×3) kernels with ReLU activation. The first layer has 32 filters with no padding, producing an output tensor of (62, 62, 32). The second layer has 64 filters, producing an output tensor of (60, 60, 64). A 2×2 max-pooling layer then reduces the size to (30, 30, 64). A dropout layer (rate = 0.2) is applied, followed by flattening to (30×30×64, 1). Finally, a dense layer with (256, 1) and ReLU activation, another dropout layer (0.2), and a softmax output layer are added. The total parameters are 14,767,818, and the classification accuracy of the testing data is 78.83%.

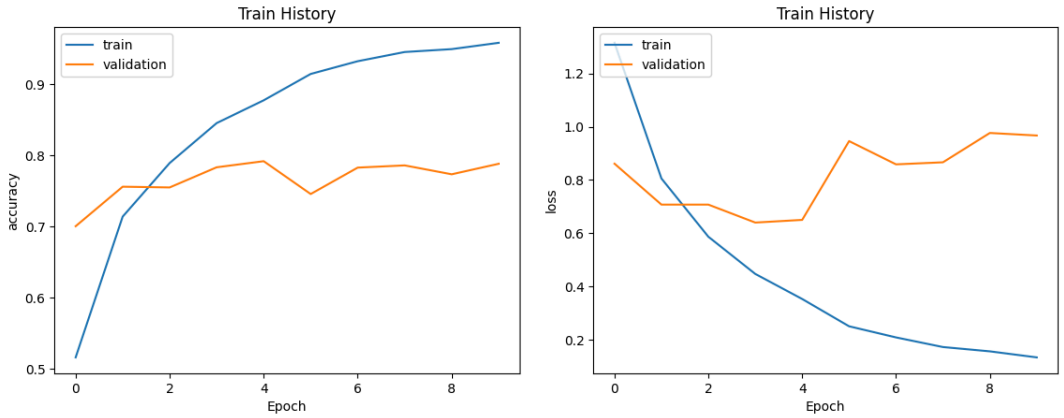


Figure 7- 8. Training and Validation Result of Model #4

Model #5: A deeper CNN architecture than Model #4, consisting of four convolution + max-pooling blocks. After flattening, a dropout layer (rate = 0.5) and two dense layers are added. Each convolution uses a 3×3 kernel with ReLU activation and doubles the number of filters after each pooling layer. The final two dense layers have output sizes (512, 1) and (10, 1), using ReLU and softmax activations respectively. The total parameters are 918,346, and the classification accuracy of the testing data is 83.01%.

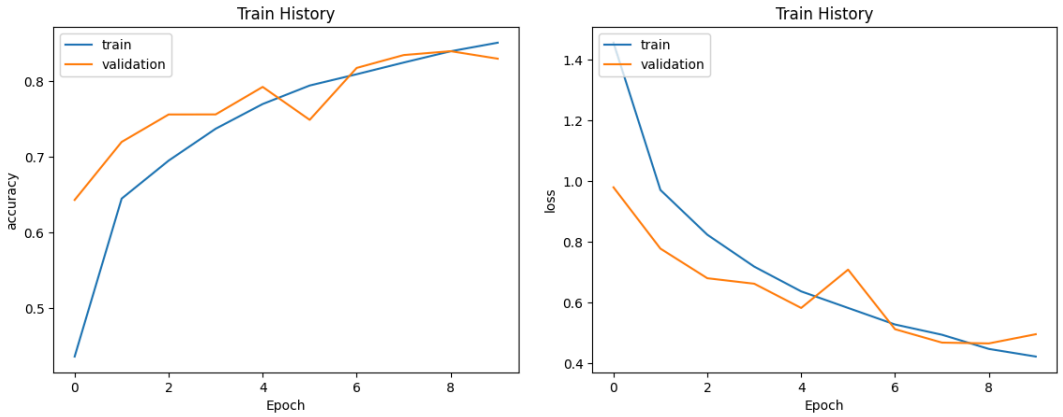


Figure 9-10. Training and Validation Result of Model #5

Model #6: Uses the same overall structure as Model #5 but replaces the convolution + pooling blocks with a VGG16 pre-trained model and applies transfer learning to improve accuracy. VGG16 also uses convolution + max-pooling layers but with more depth. Transfer learning first freezes the pre-trained layers and trains only the final fully connected (classifier) layers, then unfreezes the base model for fine-tuning with a smaller learning rate. In the first stage, 1,054,218 parameters are trained, and in the second stage, 15,768,906 parameters are fine-tuned.

The testing accuracy achieved is 93.66%.

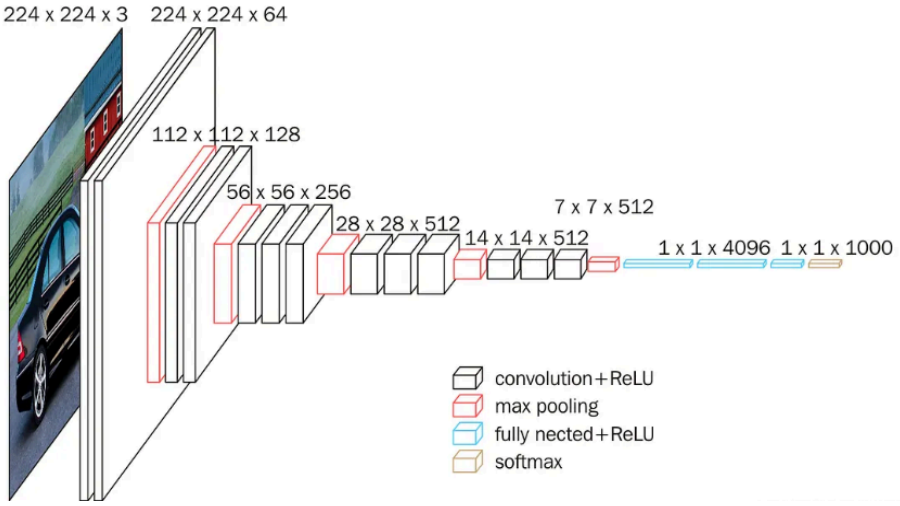


Figure11. VGG16 Architecture

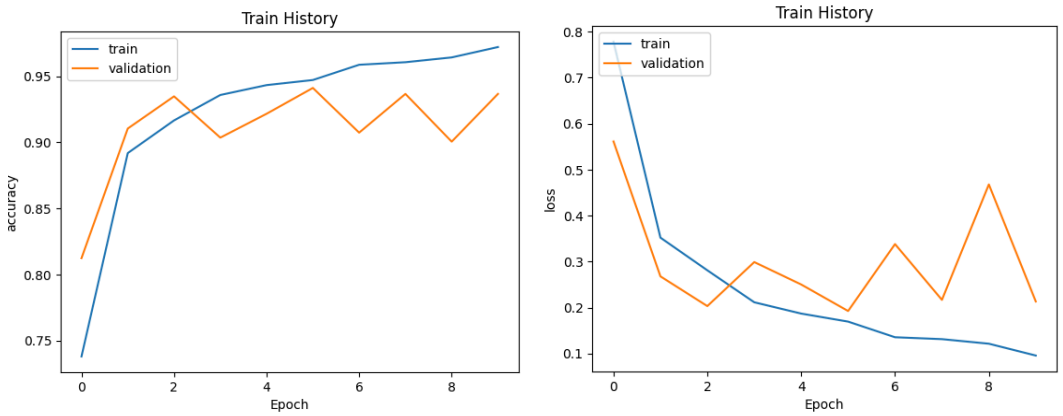


Figure 12-13. Training and Validation Result of Model #6

Model #7: Uses the same architecture as Model #6 but trains all parameters in a single step (no frozen layers) and adds a data augmentation technique to increase robustness and generalization ability. Horizontal and vertical flips are applied to enrich the dataset and help the model learn key features under different orientations. The total parameters are 15,768,906, and the classification accuracy of the testing data is 94.37%.

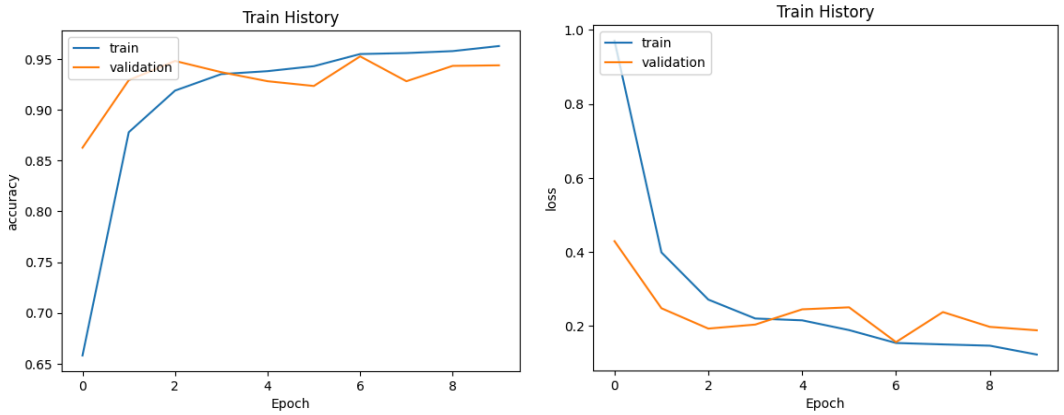


Figure 14-15. Train and Validation Result of Model #7

Model #8: Uses the same architecture as Model #7 but adds EarlyStopping and ReduceLROnPlateau (learning rate scheduling). Early stopping halts training when the monitored metric stops improving and restores the best weights to prevent overfitting. ReduceLROnPlateau lowers the learning rate when the monitored metric plateaus, helping the model approach the optimal minimum loss. The total parameters are 15,768,906, and the classification accuracy of the testing data is 94.18%.

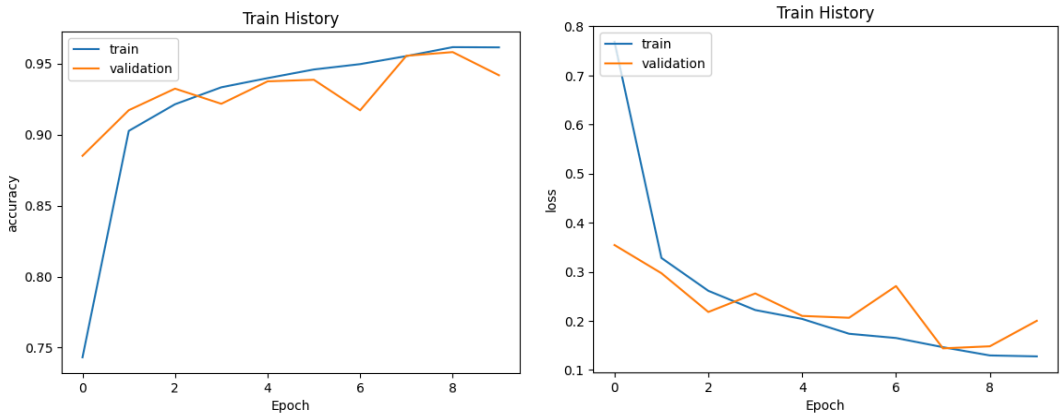


Figure 16-17. Training and Validation Result of Model #8

2. **Why the performance of Model #3 is worse than that of Models #1 and #2?**

The performance of Model #3 is worse than that of Models #1 and #2 because Model #3 is harder to converge. Although it has a deeper architecture with ReLU activation to mitigate gradient vanishing and dropout (0.2) to prevent overfitting, the model still suffers from gradient vanishing during training. Deeper models can theoretically learn more complex functions and achieve better classification performance. However, as gradients are propagated through multiple layers, they may become smaller and smaller, making the parameter updates inefficient. This causes the weights in the earlier layers to remain almost unchanged, preventing the model from converging

properly. From the training curves, we can clearly see that the training accuracy oscillates between epochs and the loss quickly plateaus around 1.9 after the first epoch. This indicates that the model's parameters are not effectively updated. In contrast, Models #1 and #2, despite having simpler architectures, manage to steadily reduce their training loss, resulting in better classification performance than Model #3.

3. **Why the performance of Model #4 is much better than that of Models #1-#3?**
   The reason Model #4 performs better than Models #1–#3 is because of its CNN architecture. CNNs have several advantages over FCNs. First, CNNs use far fewer parameters, allowing them to go deeper without suffering from gradient vanishing and making them easier to converge. Second, CNNs share weights across spatial positions, meaning the same filter is applied to every pixel. This enables CNNs to detect the same feature even when it appears in different locations, whereas FCNs must learn separate weights for each position. Third, CNNs are effective at filtering out noise and irrelevant information. Through convolution and pooling (such as max pooling or average pooling), CNNs retain the most important features while discarding less useful details. In contrast, FCNs process every pixel equally, making it harder to identify which structures are truly meaningful for classification. Because of these advantages, Model #4 converges more effectively and achieves better performance than the FCN-based Models #1–#3.
4. **What is the purpose of data augmentation used in Model #6-#8?**
   The purpose of using data augmentation is to increase the model's generalization ability and robustness, allowing it to learn which features are important for classifying or detecting objects. In Model #6, we use a VGG-based architecture with several additional layers, applying the transfer learning technique without data augmentation. Since the result reached around 94% accuracy, which is quite good for an image classification task, adding data augmentation could further increase the diversity of training samples. In Model #7, we fine-tune the entire model based on the architecture of Model #6 and introduce the data augmentation technique to enhance data variety and improve robustness. In Model #8, we build upon Model #7 by adding EarlyStopping and ReduceLROnPlateau (learning rate scheduling), which help the model achieve better performance and prevent overfitting during training.
5. **Use two different keras backbones in Model #8 and compare their performance.**
   Model #8: The total parameters are 15,768,906, and the classification accuracy of the testing data is 94.18%.
   Model #9: In this model, the ResNet50 network replaces the VGG16 model as the base architecture. The same overall network structure and data augmentation methods as in Model #8 are used. The total number of parameters increases to 27,787,658, and the classification accuracy on the testing data reaches 93.91%.

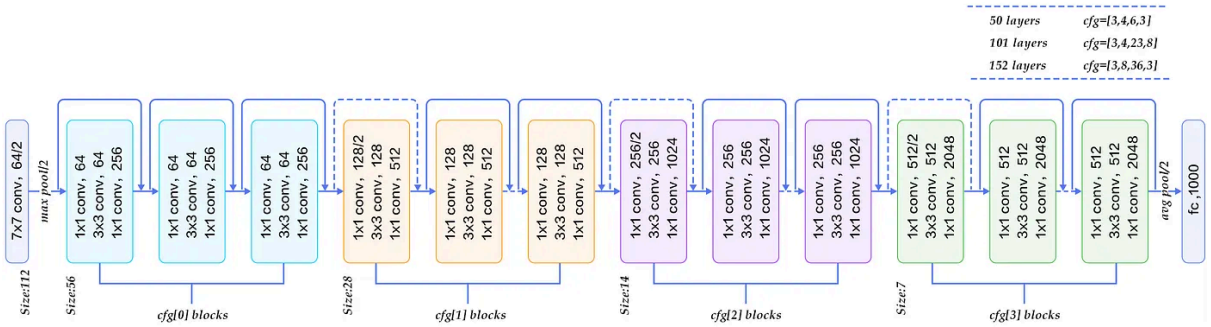


Figure 18. ResNet50 Architecture

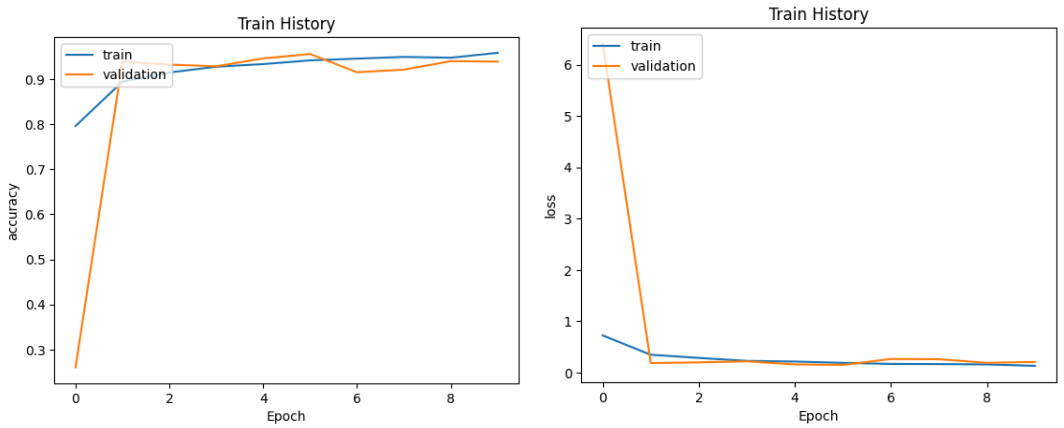


Figure 19-20. Training and Validation Result of Model #9

Model #10: In this model, the DenseNet121 architecture replaces the VGG16 base model, while keeping the same network structure and data augmentation methods as Model #8. The total number of parameters decreases to 9,140,298, and the classification accuracy on the testing data reaches 94.68%.

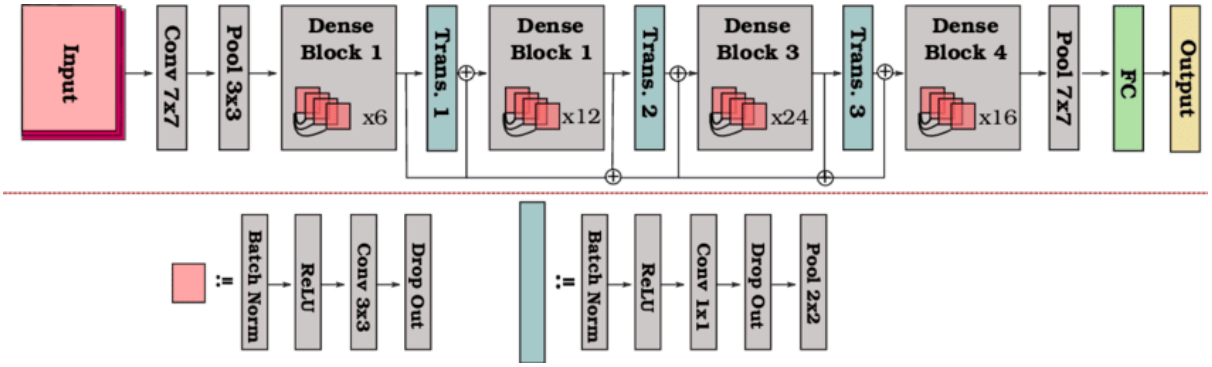


Figure 21. DenseNet121 Architecture

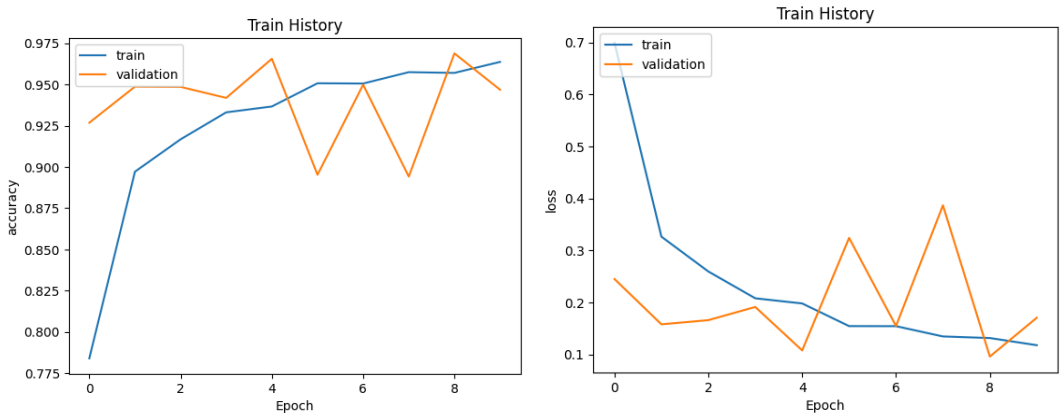


Figure 22-23. Training and Validation Result of Model #10

Comparison of Models #8–#10: All three models achieve strong performance on the classification task, with testing accuracies exceeding 90%. Among them, Model #10 (DenseNet121) achieves the highest accuracy with the fewest parameters within ten epochs of training. However, based on the training and validation curves, Model #10 appears less convergent compared to Models #8 and #9. In contrast, Model #9 (ResNet50), which uses the largest number of parameters, demonstrates the most stable and convergent training–validation behavior, even though its final testing accuracy is slightly lower. These results suggest that the choice of pre-trained model, along with data augmentation, training techniques, and hyperparameter settings, can significantly influence the final model performance.